Augustana University and Zion Clouds. Version 1.0, 24 September 2026.

Academic Advising Agent: Connector API Reference

Read-only REST API over Jenzabar J1 (J1PRD). 26 endpoints.

1. Overview
2. Authentication
3. Conventions
4. Catalog endpoints
5. Identity endpoints
6. Student endpoints
7. Advisor and administrator endpoints
8. Degree audit
9. Data handling
10. Support

1. Overview

Read-only data layer behind the Academic Advising Agent. Runs a fixed set of queries against J1 and returns JSON.

Base URL	`https://connector-api-385429559879.us-central1.run.app`
Protocol	HTTPS. All endpoints are `GET` and return JSON.
Data source	Jenzabar J1 `J1PRD`, read over the site-to-site VPN. No data cached.
Hosting	Google Cloud Run, `us-central1`, project `fit-stream-495320-f6`.
Endpoints	26: catalog (6), identity (2), student (8), advisor (9), administrator (2).

No endpoint writes to J1. The API cannot register, drop or change anything.

2. Authentication

Two layers, both required.

2.1 Google Cloud IAM

The service requires a Google-signed OIDC token. Callers without `roles/run.invoker` get `403` before reaching the application.

```
Authorization: Bearer <Google OIDC identity token>
```

To get access, supply the service account or Google identity that will call the API. It is granted `roles/run.invoker`. Tokens use the service URL as the audience; most Google client libraries do this automatically.

2.2 Identity headers

Identity-gated endpoints need two more headers. The API trusts them, so it stays behind the IAM gate and is never public.

X-User-Id	The user's J1 `ID_NUM`.
X-User-Role	`student`, `advisor` or `advising_admin`. Anything else is anonymous.

The agent derives these from the verified SSO email using `/identity/resolve`.

2.3 Roles

anonymous	Catalog and term data only.
student	Own record only. No parameter accepts another student's ID.
advisor	Own caseload and assigned advisees only. Others return `404`.
advising_admin	Institution-wide. Staff flagged `ACCESS_TYP='M'` in `ADVISING_ACCESS`.

Advisee routes verify the advisor-advisee relationship in J1 before reading data. A failed check returns `404`, not `403`, so it does not reveal whether the student exists. This runs in the API, not the agent.

3. Conventions

Terms. Two-part key: `yr` (four-digit academic year) and `trm` (`FA`, `SP`, `SU`, `IN`). The academic year starts in autumn, so `yr=2026&trm=SP` is Spring 2027. Use `/terms/resolve` rather than building keys by hand.

Parameters. `limit` is clamped on the server. Boolean flags such as `active_only` and `open_only` default to true and turn off only on an explicit `false`.

Errors. JSON body with `error` and usually `detail`.

400	`missing_param`	Required parameter absent or invalid.
401	`identity_required`	Identity headers required but not supplied.
403	`forbidden`	Role insufficient.
404	`not_found`	No such record, or advisor does not own that advisee.
500	`internal_error`	Unexpected failure. Logged on the server.

4. Catalog Endpoints

No identity headers required. No personal data.

GET /health

Service and database health. No parameters.


```
{ "status": "ok",
  "db": { "server": "100.126.140.11", "database": "J1PRD", "time": "Sep 24 2026 10:58AM" } }
```


GET /terms

Optional: `yr`, `limit`.


```
{ "count": 3,
  "terms": [ { "yr": "2026", "trm": "FA", "term_name": "Fall Semester",
               "label": "2026–2027 Year – Fall Semester",
               "begins": "2026-08-26", "ends": "2026-12-18" } ] }
```


GET /terms/resolve

Converts a phrase to a term key. Required: `q` (for example `next term`).


```
{ "interpreted": { "year": null, "term_code": null, "relative": true },
  "matches": [ { "yr": "2026", "trm": "SP", "term_name": "Spring Semester",
                 "label": "2026–2027 Year – Spring Semester",
                 "begins": "2027-01-05", "ends": "2027-05-20" } ] }
```


GET /catalog/courses

Optional: `subject`, `level`, `active_only` (default true), `limit`.


```
{ "count": 2,
  "courses": [ { "crs_cde": "BIOL 1000", "title": "Biology and Human Concerns",
                 "credit_hrs": 4, "division": "UG", "school": "CAS", "active": "A" } ] }
```


GET /catalog/sections

Required: `yr`, `trm`. Optional: `subject`, `level`, `open_only` (default true), `min_seats`, `limit`.


```
{ "count": 2, "term": { "yr": "2026", "trm": "FA" },
  "sections": [ { "yr": "2026", "trm": "FA", "crs_cde": "BIOL 1022 A", "request_num": "109881",
                  "title": "Human Anatomy Lecture & Lab", "status": "O",
                  "capacity": 40, "enrolled": 39, "seats_left": 1, "waitlisted": 0,
                  "credit_hrs": 4, "instructor_id": 714971, "instructor": "Kevin Natukunda",
                  "prerequisites_raw": null } ] }
```


GET /catalog/section

Required: `yr`, `trm`, `crs` (section code). `404` if not offered.

```
{ "section": { "yr": "2026", "trm": "FA", "crs_cde": "BIOL 1022 A", "request_num": "109881",
                "title": "Human Anatomy Lecture & Lab", "status": "O", "cancelled": "N",
                "capacity": 40, "enrolled": 39, "seats_left": 1, "waitlisted": 0,
                "credit_hrs": 4, "section_notes": "...", "instructor_id": 714971 } }
```


5. Identity Endpoints

`GET /identity/resolve`

Resolves a verified campus email to a J1 identity. Takes no `X-User-Id`. Required: `email`. `404` if no match. `role` may be `null`.


```
{ "id_num": 442933, "role": "student" }
```


`GET /me/profile`

Display name for the signed-in user. Requires `X-User-Id`. Any role.


```
{ "id": 400001, "first_name": "Taylor", "last_name": "Nelson",
  "name": "Taylor Nelson", "display_name": "Taylor" }
```

6. Student Endpoints

Scoped to `X-User-Id`. Require `X-User-Role: student`.

`GET /students/me/registration`

Registered courses with meeting times. Required: `yr`, `trm`. `status_label` decodes the raw J1 `status` to `registered`, `completed`, `dropped` or `withdrawn`.

```
{ "count": 5, "term": { "yr": "2026", "trm": "FA" },
  "registration": [ { "crs_cde": "AMSL 1120 A", "title": "Foundations in Amer Deaf Culture",
      "status": "C", "status_label": "registered", "credit_hrs": 3, "instructor": "Angela Ellman",
      "meetings": [ { "seq": 1, "days": "TR", "begin": "10:50", "end": "12:05",
                      "building_code": "MC", "building": "Madsen Center", "room": "202" } ] } ] }
```

`GET /students/me/holds`

Active holds. Students see `display_text` only where J1 marks it web-visible; advisors see it in full.

```
{ "count": 1,
  "holds": [ { "hold_cde": "SA", "hold_desc": "Registrar - Student Athlete",
               "hold_type": "Registrar Hold", "display_text": "...",
               "start_dte": "2026-08-13", "end_dte": null,
               "blocks_registration": "H", "allow_web_reg": "Y" } ] }
```

`GET /students/me/registration-window`

Add and drop windows for the student's registration group. Required: `yr`, `trm`. The `open_now` fields are evaluated at request time.

```
{ "term": { "yr": "2026", "trm": "FA" }, "found": true, "count": 1,
  "windows": [ { "group_code": "SR", "group_name": "Senior",
      "add_begin": "2026-04-13 07:00", "add_end": "2026-09-02 23:59",
      "drop_begin": "2026-04-13 07:00", "drop_end": "2026-09-09 23:59",
      "add_open_now": false, "drop_open_now": false,
      "requires_clearance": "N", "requires_advisor_approval": "N" } ] }
```

`GET /students/me/gpa`

Overall GPAs plus a breakdown by area. Separate from the per-requirement GPA in the degree audit. No parameters.

```
{ "cumulative_gpa": 3.86, "institution_gpa": 3.82, "transfer_gpa": 4.0,
  "by_area": [ { "area": "Pre-Medicine", "label": "CONC", "gpa": 3.68 } ] }
```

`GET /students/me/schedule-conflicts`

Required: `yr`, `trm`. With no other parameter, checks registered classes against each other. Add `candidate` (one section code) or `candidates` (comma-separated) to test prospective sections.

```
{ "term": { "yr": "2026", "trm": "FA" }, "candidate": null,
  "has_conflict": false, "conflicts": [],
  "current_meetings": [ { "crs_cde": "AMSL 1120 A", "seq": 1, "days": "TR",
      "begin": "10:50", "end": "12:05", "building": "Madsen Center", "room": "202" } ] }
```

GET /students/me/course-eligibility

Advisory prerequisite check. Required: `yr`, `trm`, `crs`. `status` is `met`, `not_met` or `verify`. Does not block registration.


```
{ "crs_cde": "BIOL 1022 A", "yr": "2026", "trm": "FA",
  "prerequisites_raw": null, "status": "met", "missing": [],
  "note": "No prerequisites listed.", "already_completed": { "taken": false },
  "advisory": "Advisory only. Does not block registration." }
```


GET /students/me/recommend-next-term

Courses offered in a term that satisfy an unmet requirement, based on the degree audit. Optional: `yr`, `trm` (defaults to the upcoming term).


```
{ "available": true, "term": { "yr": "2026", "trm": "FA" }, "as_of": "2026-09-23T17:50:11.427Z",
  "items": [ { "code": "AMSL 1120", "title": "Found Amer Deaf Culture",
               "satisfies": "Sign Lang Studies Minor", "kind": "minor",
               "status": "not_met", "open": true, "seats_left": 1 } ],
  "open_requirements": [ { "requirement": "31 cr. free electives", "kind": "electives",
                           "count_needed": 0, "reason": "open_electives" } ],
  "note": "From the degree audit, not a forward term-by-term plan." }
```


GET /students/me/degree-progress

Live degree audit. Optional flags `detail`, `checklist`, `rules`. See section 8.

7. Advisor and Administrator Endpoints

`GET /advisors/me/caseload`

Caseload for a term. Required: `yr`, `trm`. Optional `status=unregistered`. Rows carry registration and hold counts, so caseload-wide questions need one call, not one per advisee.

```
{ "count": 9, "term": { "yr": "2026", "trm": "FA" }, "status": "overview",
  "advisees": [ { "advisee_id": 400001, "advisee_name": "Taylor Nelson",
                  "registered_count": 6, "registered_credits": 13, "active_holds": 2 } ] }
```

For an administrator the response also carries `"institution_wide": true` and `total_students`.

`GET /advisors/me/advisees/{adviseeId}/...`

Eight routes mirror the student endpoints for one advisee. Each returns the same body plus `advisee_id`. All run the ownership check first.

Path suffix	Equivalent student endpoint
`/registration`	`/students/me/registration`
`/holds`	`/students/me/holds`
`/registration-window`	`/students/me/registration-window`
`/gpa`	`/students/me/gpa`
`/schedule-conflicts`	`/students/me/schedule-conflicts`
`/course-eligibility`	`/students/me/course-eligibility`
`/recommend-next-term`	`/students/me/recommend-next-term`
`/degree-progress`	`/students/me/degree-progress`

```
GET /advisors/me/advisees/400001/gpa

{ "advisee_id": "400001", "cumulative_gpa": 3.86, "institution_gpa": 3.82,
  "transfer_gpa": 4.0, "by_area": [ ... ] }
```

`GET /students/search`

Administrator only. Name search returning `id_num` for use with advisee routes. Required: `q` (minimum two characters). Optional `limit` (default 25, max 50).

```
{ "count": 3, "query": "smith",
  "students": [ { "id_num": 400002, "name": "Jordan Smith" } ] }
```

`GET /advisors/me/caseload?scope=all`

Administrator only. Full institutional roster, paged. Required: `yr`, `trm`, `scope=all`. Optional `limit` (default 100, max 500) and `offset`. `total` is the unpaged count.

```
{ "count": 2, "term": { "yr": "2026", "trm": "FA" }, "scope": "all",
  "total": 3424, "limit": 2, "offset": 0,
  "advisees": [ { "advisee_id": 400003, "advisee_name": "Alex Miller",
                  "registered_count": 6, "registered_credits": 16, "active_holds": 1 } ] }
```

8. Degree Audit

Returned by `/students/me/degree-progress` and the advisee equivalent. Three optional flags add sections.

detail=true	Full requirement tree under each programme.
checklist=true	`audit.checklist` : one entry per requirement with status, need, have, completed courses and remaining options.
rules=true	`degree_rules` : graduation rules and course repeats. Undergraduates only; graduate students get `grad_rules` .


```
{ "declared": { "current_degree": "Y", "degree_code": "BA", "major": "BIOL",
                "major_2": null, "major_3": null, "concentration": "PMED",
                "catalog_year": "2024", "expected_grad_year": "2026" },
  "audit": {
    "available": true, "as_of": "2026-09-23T17:50:11.427Z", "dual_career": false,
    "summary": { "programs_total": 6, "programs_met": 0, "programs_in_progress": 0 },
    "gen_ed": { "curriculum": "Sophia Core", "aim_label": "CORE", "met": "not_met",
                "hrs_earned": 61, "hrs_needed": 45 },
    "programs": [ { "aim_label": "CONC", "code": "PMED", "description": "Pre-Medicine",
        "met": "not_met", "met_code": "N", "hrs_earned": 47, "hrs_needed": 54, "hrs_max": 55,
        "hrs_in_progress": 0, "gpa_cur": 3.68, "gpa_needed": 0,
        "count_actual": 12, "count_min": 0,
        "error_code": "HOURS", "error_label": "needs more credit hours",
        "credits_met": false, "incomplete_requirements": [] } ],
    "total_credits": { "earned": 127, "needed": 124, "met": true,
                       "requirements_complete": false, "note": "..." },
    "overall_gpa": { "cumulative": 3.86, "institution": 3.82, "transfer": 4.0 } },
  "plan": { "available": false, "reason": "AVStudentPlan not populated at AugU" } }
```


Two fields are easy to misread:

- `credits_met` covers only the credit count. A programme can be `not_met` while `credits_met` is true. Check `incomplete_requirements` before calling it complete.
- `total_credits.met` means earned is at least needed. If `requirements_complete` is false, the degree is not finished.

`plan` is always unavailable. J1 holds no forward term-by-term plans at Augustana.

8.1 degree_rules

Graduation rules with departmental overrides, plus course-repeat tracking. Advisory only.

```
{ "applies": true, "major": "BIOL", "department_override": "BIOL",
  "min_grade": { "rule": "C- in ALL major courses",
                 "below_c_minus": [ { "scope": "major", "course": "BIOL 3900 A",
                                      "grade": "D+", "action": "retake_to_count" } ] },
  "residency": { "major_courses_total": 11, "at_augustana": 11, "transfer": 0,
                 "pct_at_augustana": 1, "meets_50": true },
  "degree": { "cum_gpa": 3.86, "min_cum_gpa": 2.0, "cum_gpa_ok": true },
  "retake": { "policy": "university", "repeated_courses": 0, "courses": [], "detail": [],
              "major": { "used": 0, "limit": null, "remaining": null, "courses": [] },
              "other": { "used": 0, "limit": null, "courses": [] },
              "max_repeats_per_course": null, "courses_over_repeat_limit": [],
              "at_or_over_limit": false, "note": "...", "source": "..." },
  "notes": [ "..." ],
  "advisory": "Advisory only. Never blocks registration or graduation." }
```


Repeat counts come from J1 repeat flags, not from duplicate course codes, so a course taken in several terms is not a repeat and a renumbered course counts once. `policy` is `university` for most majors, where any course may be repeated and `limit` is `null`. Nursing has caps and returns `policy: "nursing"` with `nursing` and `prereq` buckets instead of `major`. If no audit tree exists, `major.available` is `false` rather than zero.

9. Data Handling

- No student data is copied or cached. Every response is read from J1 at request time.
- J1 is reached over the site-to-site VPN on private addressing. Database traffic does not cross the public internet.
- Credentials are held in Google Secret Manager and mounted at runtime. They are in no source file, image or state file.
- Student scoping and advisee ownership are enforced in the API on every request.
- The API does not read `BIOGRAPH_MASTER`, address or telephone tables, or free-text notes.
- The database account is read-only at the database level.

10. Support

- **Access.** Supply the service account or Google identity to be granted `roles/run.invoker`.
- **Stability.** These endpoints and shapes are stable. Fields and endpoints may be added. Breaking changes will be agreed in advance.
- **Issues.** Include endpoint, parameters, approximate time, and the `error` and `detail` returned.
- **Monitoring.** `/health` confirms service and database reachability.

Verified against the production service on 24 September 2026. Example payloads are accurate in structure. Identifiers and names in student examples are illustrative.